



S.E.P.

TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE TUXTEPEC

ASIGNATURA:
CIENCIA DE DATOS

DOCENTE:
JULIO AGUILAR CARMONA

ACTIVIDAD:
PRACTICA PYTHON

ALUMNO:
Urieta Ubieta Jesus – 22350384

(NOTA: CREE UN ENTORNO VIRTUAL EN EL QUE INSTALE LAS LIBRERIAS Y GUARDE MIS PRACTICAS EN LA MISMA CARPETA)

PRACTICA 2: Análisis de Calidad Industrial (Industria 4.0)

PASO 1: Creamos El Archivo Para La Practica 3 Y Creamos El Código Para Poner A Prueba Las Habilidades De Manipulación De Datos Con Dplyr, Cálculo De Medidas Estadísticas Y Visualización Con Ggplot2 Para Generar Interpretaciones De Negocio.

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Dataset sintético
np.random.seed(42)
n = 500
data = pd.DataFrame({
    "temperatura": np.random.normal(75, 5, n),
    "presion": np.random.normal(30, 3, n),
    "tasa_error": np.random.normal(0.05, 0.01, n),
    "turno": np.random.choice(["Matutino", "Vespertino"], n)
})

# Estadísticos
print("Media:", data["temperatura"].mean())
print("Mediana:", data["temperatura"].median())
print("Desviación estándar:", data["temperatura"].std())

# Correlación
print(data[["temperatura", "tasa_error"]].corr())

# Comparación por turnos
print(data.groupby("turno")["tasa_error"].mean())

# Boxplot
sns.boxplot(x="turno", y="tasa_error", data=data)
plt.show()

# Scatter con regresión
sns.lmplot(x="temperatura", y="tasa_error", data=data)
plt.show()
```

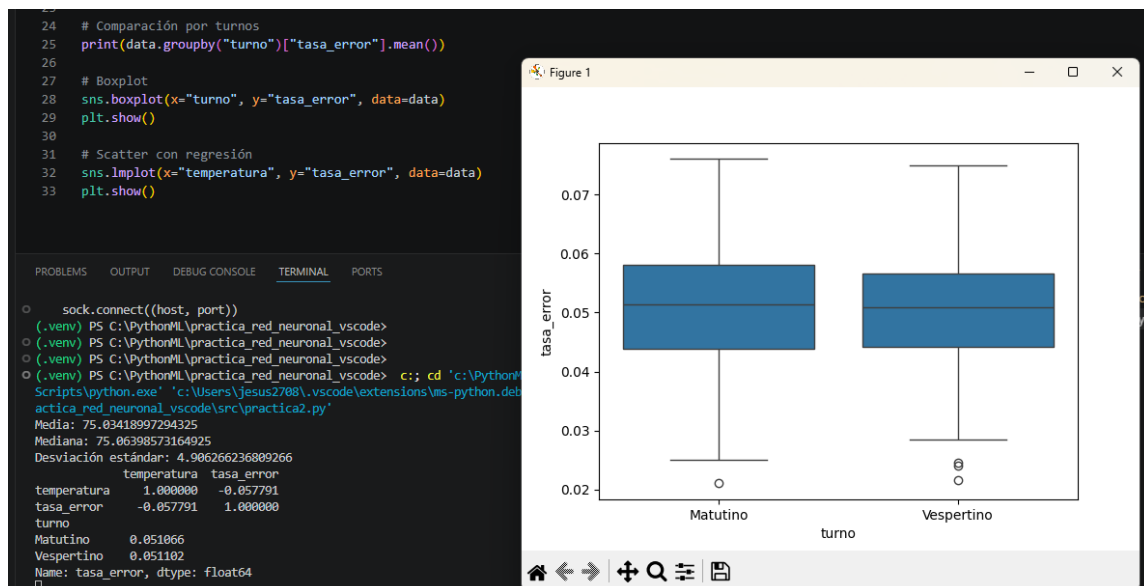
PASO 2, Instalamos Las Librerias

```
14 })
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
(.venv) PS C:\PythonML\practica_red_neuronal_vscode> python -m pip install pandas numpy matplotlib seaborn scikit-learn
Collecting pandas
  Downloading pandas-2.3.3-cp310-cp310-win_amd64.whl.metadata (19 kB)
(.venv) PS C:\PythonML\practica_red_neuronal_vscode> python -m pip install pandas numpy matplotlib seaborn scikit-learn
Collecting pandas
  Downloading pandas-2.3.3-cp310-cp310-win_amd64.whl.metadata (19 kB)
Requirement already satisfied: numpy in .\venv\lib\site-packages (2.2.6)
Requirement already satisfied: matplotlib in .\venv\lib\site-packages (3.10.9)
Collecting seaborn
  Downloading seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Collecting scikit-learn
  Using cached scikit_learn-1.7.2-cp310-cp310-win_amd64.whl.metadata (11 kB)
Requirement already satisfied: python-dateutil>=2.8.2 in .\venv\lib\site-packages (from pandas) (2.9.0.post0)
Collecting pytz>=2020.1 (from pandas)
  Downloading pytz-2026.2-py2.py3-none-any.whl.metadata (22 kB)
Collecting tzdata>=2022.7 (from pandas)
  Downloading tzdata-2026.2-py2.py3-none-any.whl.metadata (1.4 kB)
Requirement already satisfied: contourpy>=1.0.1 in .\venv\lib\site-packages (from matplotlib) (1.3.2)
Requirement already satisfied: cycler>=0.10 in .\venv\lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in .\venv\lib\site-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in .\venv\lib\site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in .\venv\lib\site-packages (from matplotlib) (26.2)
Requirement already satisfied: pillow>=8 in .\venv\lib\site-packages (from matplotlib) (12.2.0)
Requirement already satisfied: pyparsing>=3 in .\venv\lib\site-packages (from matplotlib) (3.3.2)
Collecting scipy>=1.8.0 (from scikit-learn)
  Using cached scipy-1.15.3-cp310-cp310-win_amd64.whl.metadata (60 kB)
Collecting joblib>=1.2.0 (from scikit-learn)
  Using cached joblib-1.5.3-py3-none-any.whl.metadata (5.5 kB)
Collecting threadpoolctl>=3.1.0 (from scikit-learn)
  Using cached threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)
Requirement already satisfied: six>=1.5 in .\venv\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Downloading pandas-2.3.3-cp310-cp310-win_amd64.whl (11.3 MB)
11.3/11.3 MB 1.7 MB/s 0:00:07
Downloading seaborn-0.13.2-py3-none-any.whl (294 kB)
Using cached scikit_learn-1.7.2-cp310-cp310-win_amd64.whl (8.9 MB)
Using cached joblib-1.5.3-py3-none-any.whl (309 kB)
Downloading pytz-2026.2-py2.py3-none-any.whl (510 kB)
Using cached scipy-1.15.3-cp310-cp310-win_amd64.whl (41.3 MB)
Using cached threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
Downloading tzdata-2026.2-py2.py3-none-any.whl (349 kB)
Installing collected packages: pytz, tzdata, threadpoolctl, scipy, joblib, scikit-learn, pandas, seaborn
Successfully installed joblib-1.5.3 pandas-2.3.3 pytz-2026.2 scikit-learn-1.7.2 scipy-1.15.3 seaborn-0.13.2 threadpoolctl-3.6.0 tzdata-2026.2
```

PASO 3 Ejecutamos



```

23
24 # Comparación por turnos
25 print(data.groupby("turno")["tasa_error"].mean())
26
27 # Boxplot
28 sns.boxplot(x="turno", y="tasa_error", data=data)
29 plt.show()
30
31 # Scatter con regresión
32 sns.lmplot(x="temperatura", y="tasa_error", data=data)
33 plt.show()

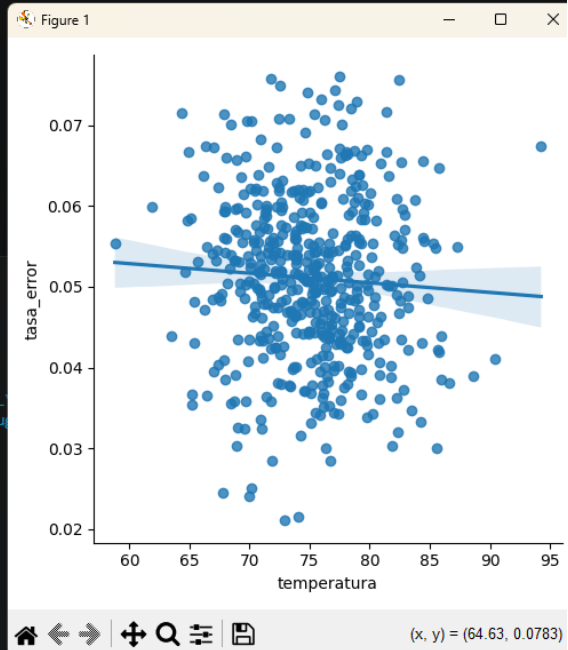
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

○ sock.connect((host, port))
(.venv) PS C:\PythonML\practica_red_neuronal_vscode>
○ (.venv) PS C:\PythonML\practica_red_neuronal_vscode>
○ (.venv) PS C:\PythonML\practica_red_neuronal_vscode>
○ (.venv) PS C:\PythonML\practica_red_neuronal_vscode> c;; cd 'c:\PythonML\
Scripts\python.exe' 'c:\Users\jesus2708\.vscode\extensions\ms-python.debu
actica_red_neuronal_vscode\src\practica2.py'
Media: 75.03418997294325
Mediana: 75.06398573164925
Desviación estándar: 4.906266236809266
temperatura tasa_error
temperatura 1.000000 -0.057791
tasa_error -0.057791 1.000000
turno
Matutino 0.051066
Vespertino 0.051102
Name: tasa_error, dtype: float64

```



PRACTICA 3: Análisis de Eficiencia en Logística Global

PASO 1: CREAMOS EL ARCHIVO PARA LA PRACTICA 3 Y CREAMOS EL CODIGO PARA VALIDAR DE FORMA INDEPENDIENTE LA CAPACIDAD DE LIMPIAR DATOS, REALIZAR ANÁLISIS EXPLORATORIO, AGRUPAR INFORMACIÓN Y APLICAR PRUEBAS DE HIPÓTESIS PARA LA TOMA DE DECISIONES BASADA EN DATOS.

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.cluster import KMeans
from scipy.stats import ttest_ind

# Dataset sintético
np.random.seed(123)
n = 600
logistica = pd.DataFrame({
    "distancia_km": np.random.normal(300, 50, n),
    "tiempo_entrega_hrs": np.random.normal(20, 5, n),
    "tipo_transporte": np.random.choice(["Terrestre", "Aéreo"], n),
    "costo": np.random.normal(500, 100, n)
})

# Imputación de NA con mediana
logistica.loc[np.random.choice(n, 20), "distancia_km"] = np.nan
logistica["distancia_km"].fillna(logistica["distancia_km"].median(), inplace=True)

# Descriptivos
print(logistica["tiempo_entrega_hrs"].mean(), logistica["tiempo_entrega_hrs"].std())

# Correlación
print(logistica[["distancia_km", "tiempo_entrega_hrs"]].corr())

# Agrupación
print(logistica.groupby("tipo_transporte")[["tiempo_entrega_hrs", "costo"]].mean())

# Prueba t-test
from scipy.stats import ttest_ind
```

```
terrestre = logistica[logistica["tipo_transporte"]=="Terrestre"]["tiempo_entrega_hrs"]
aereo = logistica[logistica["tipo_transporte"]=="Aéreo"]["tiempo_entrega_hrs"]
print(ttest_ind(terrestre, aereo))
```

Visualizaciones

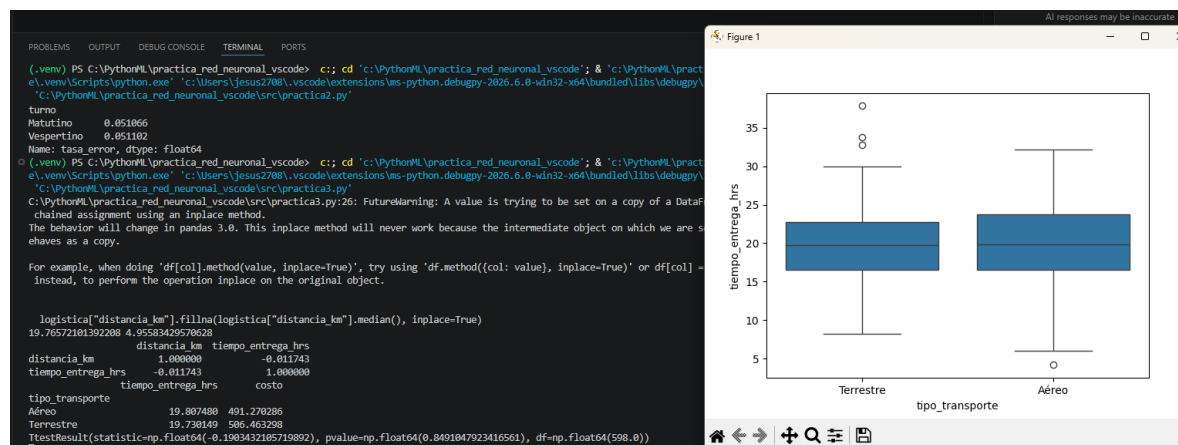
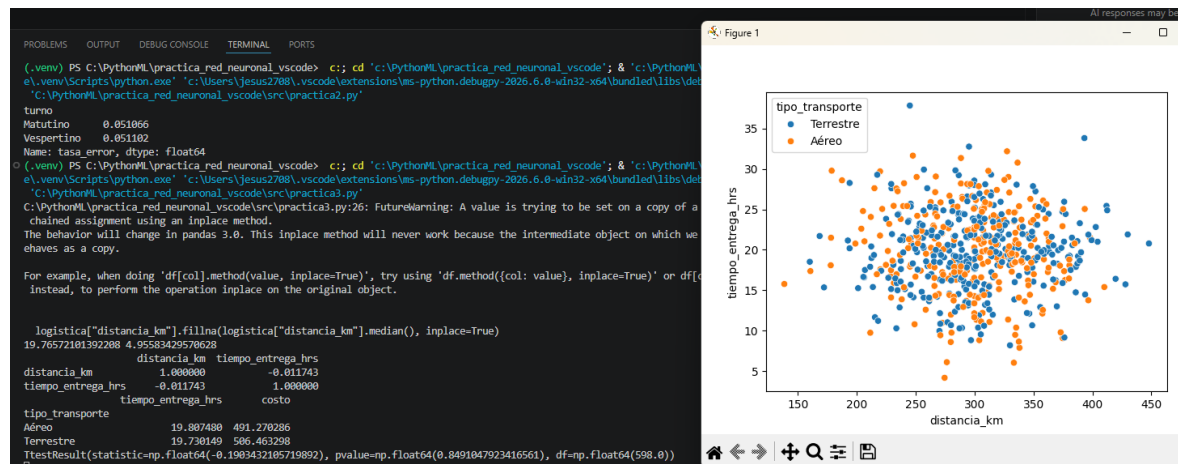
```
sns.scatterplot(x="distancia_km", y="tiempo_entrega_hrs", hue="tipo_transporte",
data=logistica)
plt.show()
```

```
sns.boxplot(x="tipo_transporte", y="tiempo_entrega_hrs", data=logistica)
plt.show()
```

PASO 2: INSTALAMOS LAS LIBRERIAS SI SON NECESARIAS DE NUEVO

PASO 3: GENERAMOS EL DATASET

PASO 4: EJECUTAMOS



PRACTICA 4: Reducción de Dimensionalidad en Monitoreo Industrial

PASO 1: CREAMOS EL ARCHIVO PARA LA PRACTICA Y CREAMOS EL CODIGO AÑADIENDO LOS IMPORTS

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Dataset sintético
np.random.seed(42)
n = 200
datos = pd.DataFrame({
    "temp_nucleo": np.random.normal(100, 5, n),
    "presion_interna": np.random.normal(50, 10, n),
    "vibracion_motor": np.random.normal(20, 2, n),
    "flujo_gas": np.random.normal(20, 2, n),
    "oxigeno": np.random.normal(15, 1, n),
    "co2": np.random.normal(25, 3, n),
    "humedad": np.random.uniform(30, 40, n),
    "voltaje": np.random.normal(220, 2, n),
    "corriente": np.random.normal(15, 0.5, n),
    "ruido_db": np.random.normal(60, 5, n),
    "eficiencia": np.random.normal(80, 5, n)
})

# Escalamiento
scaler = StandardScaler()
scaled = scaler.fit_transform(datos)

# PCA
pca = PCA()
pca.fit(scaled)

print("Varianza acumulada:", np.cumsum(pca.explained_variance_ratio_))

plt.plot(np.cumsum(pca.explained_variance_ratio_))
plt.xlabel("Componentes")
plt.ylabel("Varianza acumulada")
plt.show()
```

PASO 2: INSTALAMOS LAS LIBRERIAS SI SON NECESARIAS DE NUEVO

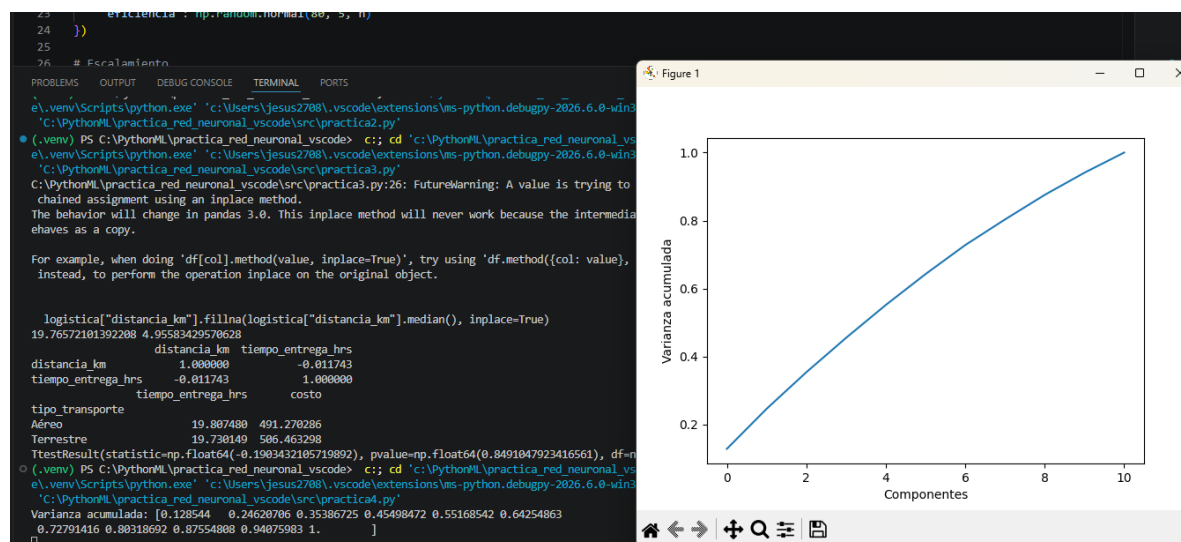
PASO 3: GENERAMOS EL DATASET

PASO 4: GENERAR EL SCREE PLOT

PASO 5: EJECUTAMOS

PASO 6: DETERMINAR CUÁNTOS COMPONENTES ALCANZAN EL 85% DE VARIANZA ACUMULADA E IDENTIFICAR QUÉ SENSORES TIENEN MÁS PESO EN EL PRIMER COMPONENTE

```
(.venv) PS C:\PythonML\practica_red_neuronal_vscode> c:; cd 'c:\PythonML\practica_red_neuronal_vscode\Scripts\python.exe' 'c:\Users\jesus2708\.vscode\extensions\ms-python.debugpy-2026.6.0-win32\bin\python.exe' 'C:\PythonML\practica_red_neuronal_vscode\src\practica4.py'  
Varianza acumulada: [0.128544 0.24620706 0.35386725 0.45498472 0.55168542 0.64254863  
0.72791416 0.80318692 0.87554808 0.94075983 1. ]
```



PRACTICA 5: Reducción de Dimensiones en Tráfico de Red (Cyber-Systems)

PASO 1: CREAMOS EL ARCHIVO Y CODIGO PARA LA PRACTICA Y AÑADIMOS LOS IMPORTS numpy, pandas, matplotlib, standarscaler y PCA

PASO 2: GENERAMOS EL DATASET DE METRICAS Y CREAMOS LAS VARIABLES DEPENDIENTES

```
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

np.random.seed(123)
n = 300
datos_red = pd.DataFrame({
    "duracion_ms": np.random.normal(50, 10, n),
    "paquetes_enviados": np.random.normal(100, 20, n),
    "errores_checksum": np.random.poisson(2, n),
    "latencia_avg": np.random.normal(15, 5, n),
    "jitter": np.random.normal(2, 0.5, n),
    "uso_memoria_sw": np.random.normal(40, 10, n),
    "peticiones_http": np.random.normal(200, 50, n)
})

# Variables dependientes
datos_red["bytes_enviados"] = datos_red["paquetes_enviados"]*1500 +
np.random.normal(0,500,n)
datos_red["reintentos_tcp"] = datos_red["errores_checksum"]*1.5 + np.random.normal(0,0.5,n)
datos_red["carga_cpu_router"] = datos_red["paquetes_enviados"]*0.4 +
datos_red["latencia_avg"]*0.2

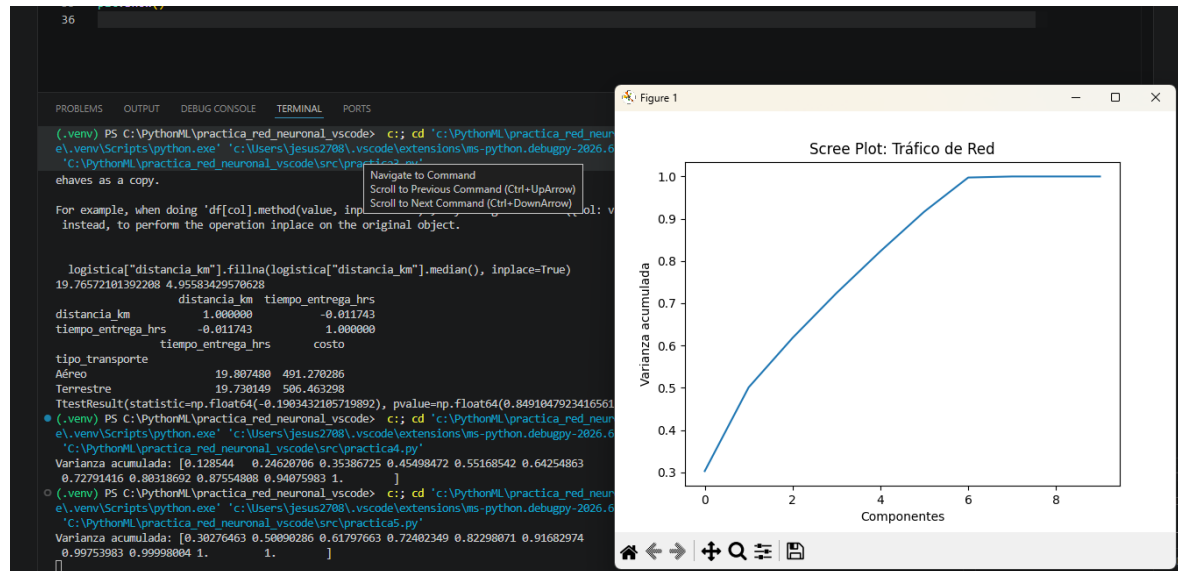
# PCA
scaled = StandardScaler().fit_transform(datos_red)
pca = PCA()
pca.fit(scaled)

print("Varianza acumulada:", np.cumsum(pca.explained_variance_ratio_))

plt.plot(np.cumsum(pca.explained_variance_ratio_))
plt.xlabel("Componentes")
plt.ylabel("Varianza acumulada")
plt.title("Scree Plot: Tráfico de Red")
plt.show()
```

APLICAMOS EL PCA Y EJECUTAMOS

```
(.venv) PS C:\PythonML\practica_red_neuronal_vscode> c;; cd 'c:\PythonML\practica_red_neuronal_vscode'; & 'c:\PythonML\practica_red_neuronal_vscode\Scripts\python.exe' 'c:\Users\jesus2708\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' 'c:\PythonML\practica_red_neuronal_vscode\src\practica5.py'
Varianza acumulada: [0.30276463 0.50090286 0.61797663 0.72402349 0.82298071 0.91682974
0.99753983 0.99998004 1.         1.         ]
(.venv) PS C:\PythonML\practica_red_neuronal_vscode> |
```



PRACTICA 6: Optimización de Sensores en Smart Cities" subtitle: "Unidad 2: Tratamiento de Datos - TecNM" author: "Guía Paso a Paso con Interpretación" output: html_document: toc: true theme: united

PASO 1: CREAMOS EL ARCHIVO Y CODIGO PARA LA PRACTICA Y AÑADIMOS LOS IMPORTS numpy, pandas, matplotlib, standarscaler y PCA

PASO 2: GENERAMOS EL DATASET URBANO Y CREAMOS LAS VARIABLES DEPENDIENTES co2_ppm, no2_ppb, particulas_pm25, nivel_ruido_db

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

np.random.seed(456)
n = 400
datos_urbanos = pd.DataFrame({
    "temp_ambiente": np.random.normal(28, 4, n),
    "humedad_rel": np.random.normal(60, 10, n),
    "densidad_vehiculos": np.random.normal(120, 30, n),
    "velocidad_viento": np.random.normal(15, 5, n),
    "radiacion_solar": np.random.normal(800, 100, n),
    "conteo_peatones": np.random.normal(50, 15, n)
})

# Variables dependientes
datos_urbanos["co2_ppm"] = datos_urbanos["densidad_vehiculos"]*3.5 +
np.random.normal(300,50,n)
datos_urbanos["no2_ppb"] = datos_urbanos["co2_ppm"]*0.1 + np.random.normal(5,2,n)
datos_urbanos["particulas_pm25"] = datos_urbanos["co2_ppm"]*0.05 +
np.random.normal(10,3,n)
datos_urbanos["nivel_ruido_db"] = datos_urbanos["densidad_vehiculos"]*0.2 + 50 +
np.random.normal(0,5,n)

scaled = StandardScaler().fit_transform(datos_urbanos)
pca = PCA()
pca.fit(scaled)

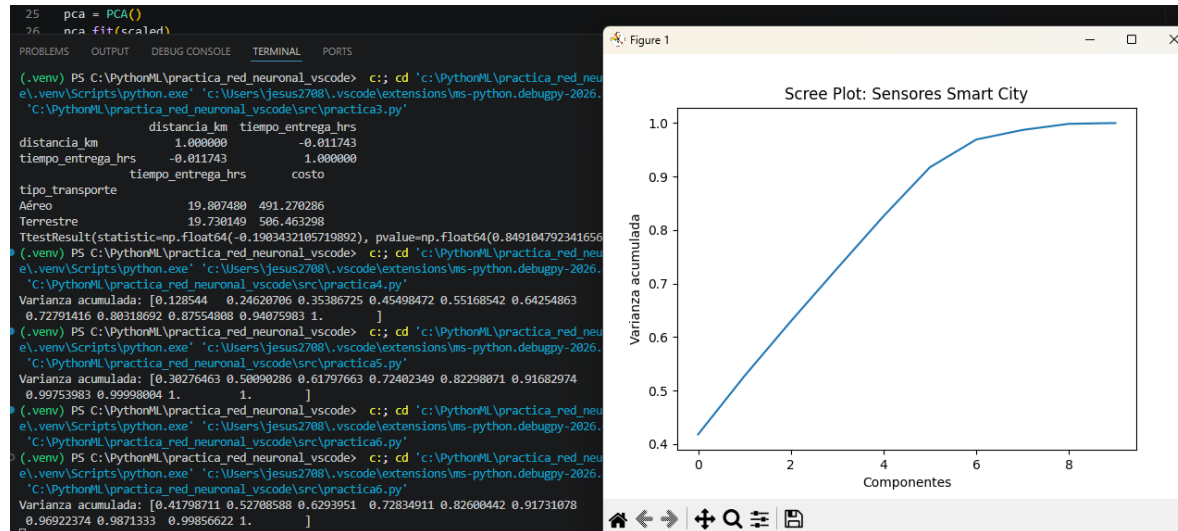
print("Varianza acumulada:", np.cumsum(pca.explained_variance_ratio_))

plt.plot(np.cumsum(pca.explained_variance_ratio_))
plt.xlabel("Componentes")
plt.ylabel("Varianza acumulada")
```

```
plt.title("Scree Plot: Sensores Smart City")
plt.show()
```

PASO 3: EJECUTAMOS

```
C:\PythonML\practica_red_neuronal_vscode\src\practica6.py
(.venv) PS C:\PythonML\practica_red_neuronal_vscode> c;; cd 'c:\PythonML\practica_red_neuronal_vscode'; & 'c:\PythonML\practica_red_neuronal_vscode\src\practica6.py'
e\.venv\Scripts\python.exe' 'c:\Users\jesus2708\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\python.exe' 'c:\Users\jesus2708\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\python.exe' 'c:\PythonML\practica_red_neuronal_vscode\src\practica6.py'
Varianza acumulada: [0.41798711 0.52708588 0.6293951 0.72834911 0.82600442 0.91731078
0.96922374 0.9871333 0.99856622 1. ]
```



PRACTICA 7: Implementación de Modelos de Machine Learning

PASO 1: CREAMOS EL ARCHIVO Y CODIGO PARA LA PRACTICA Y AÑADIMOS LOS IMPORTS numpy, pandas, matplotlib, seaborn, train_test_split, LinearRegression, KNeighborsClassifier, KMeans

PASO 2: GENERAMOS EL DATASET DE EMPLEADOS Y QUE CALCULE salario Y LA retencion

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.cluster import KMeans

# Dataset sintético
np.random.seed(789)
n = 500
df_empleados = pd.DataFrame({
    "experiencia": np.random.normal(5, 2, n),
    "certificaciones": np.random.poisson(3, n),
    "habilidades_sociales": np.random.uniform(1, 10, n),
    "remoto": np.random.choice([0,1], n)
})
df_empleados["salario"] = 25000 + df_empleados["experiencia"]*5000 +
df_empleados["certificaciones"]*2000 + np.random.normal(0,3000,n)
df_empleados["retencion"] = (df_empleados["salario"]>30000).astype(int)

# --- Regresión Lineal ---
X = df_empleados[["experiencia","certificaciones"]]
y = df_empleados["salario"]
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)
modelo = LinearRegression().fit(X_train,y_train)
print("Coeficientes:", modelo.coef_)
print("Intercepto:", modelo.intercept_)

# --- Clasificación KNN ---
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, (y_train>30000).astype(int))
print("Accuracy KNN:", knn.score(X_test,(y_test>30000).astype(int)))
```

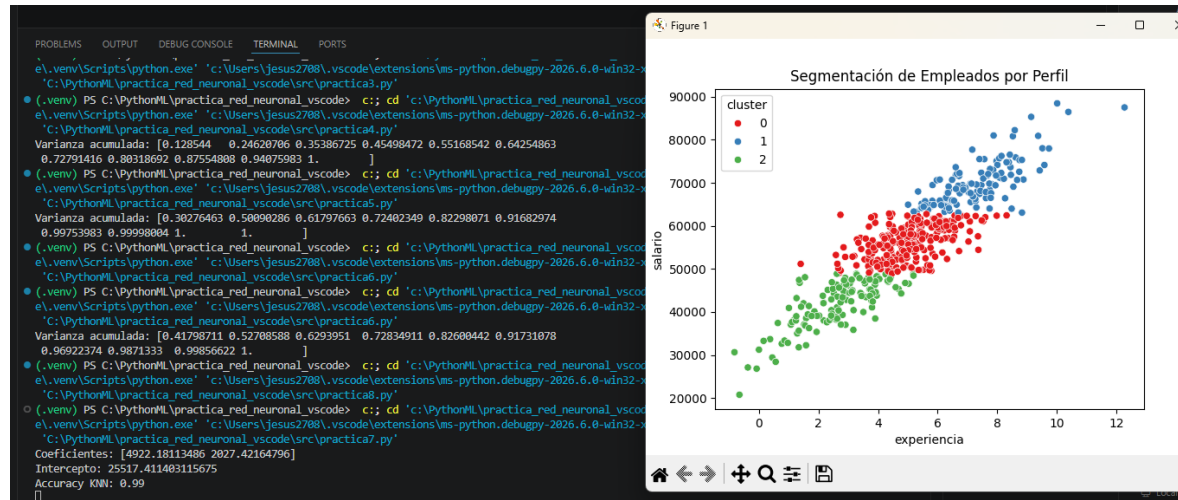
```
# --- Clustering K-Means ---
datos_clustering = df_empleados[["experiencia", "salario"]].values
kmeans = KMeans(n_clusters=3, random_state=42).fit(datos_clustering)
df_empleados["cluster"] = kmeans.labels_

sns.scatterplot(x="experiencia", y="salario", hue="cluster", data=df_empleados,
palette="Set1")
plt.title("Segmentación de Empleados por Perfil")
plt.show()
```

PASO 3: APLICAMOS LA REGRESION LINEAL PARA GRAFICAR LA SEGMENTACION DE LOS EMPLEADOS

PASO 4: EJECUTAMOS

```
(.venv) PS C:\PythonML\practica_red_neuronal_vscode> c:: cd 'c:\PythonML\practica_red_neuronal_vscode\Scripts\python.exe' 'c:\Users\Jesus2708\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bin\python.exe' 'C:\PythonML\practica_red_neuronal_vscode\src\practica7.py'
Coeficientes: [4922.18113486 2027.42164796]
Intercepto: 25517.411403115675
Accuracy KNN: 0.99
```



PRACTICA 8: Clasificación de Intrusiones con KNN y Logística

PASO 1: CREAMOS EL ARCHIVO Y CODIGO PARA LA PRACTICA Y AÑADIMOS LOS IMPORTS numpy, pandas, matplotlib, seaborn, train_test_split, LinearRegression, KNeighborsClassifier, KMeans

PASO 2: GENERAMOS EL DATASET DE latencia, intentos fallidos Y EL tamaño del paquete Y QUE CREE LA ETIQUETA DE es_ataque

PASO 3: APLICAMOS LA REGRESION LOGISTICA, LA CLASIFICACION KNN Y EL CLUSTERING KMenas

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.cluster import KMeans

# Dataset sintético
np.random.seed(444)
n = 400
df_red = pd.DataFrame({
    "latencia": np.random.normal(20, 5, n),
    "intentos_fallidos": np.random.poisson(1, n),
    "tamano_paquete": np.random.normal(500, 100, n)
})

# Etiqueta de ataque
df_red["es_ataque"] = ((df_red["intentos_fallidos"] > 2) | (df_red["latencia"] > 35)).astype(int)

# --- Regresión Logística ---
X = df_red[["latencia", "intentos_fallidos", "tamano_paquete"]]
y = df_red["es_ataque"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

modelo_log = LogisticRegression().fit(X_train, y_train)
y_pred = modelo_log.predict(X_test)
print("Accuracy Logística:", accuracy_score(y_test, y_pred))

# --- Clasificación KNN ---
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
```

```
print("Accuracy KNN:", knn.score(X_test,y_test))

# --- Clustering K-Means ---
kmeans = KMeans(n_clusters=2, random_state=111).fit(X)
df_red["cluster"] = kmeans.labels_
print(pd.crosstab(df_red["es_ataque"], df_red["cluster"]))
```

PASO 4: EJECUTAMOS

```
● (.venv) PS C:\PythonML\practica_red_neuronal_vscode> c:: cd
e\venv\Scripts\python.exe' 'c:\Users\jesus2708\.vscode\exter
'C:\PythonML\practica_red_neuronal_vscode\src\practica8.py'
Accuracy Logística: 1.0
Accuracy KNN: 0.95
cluster      0      1
es_ataque
0           178    191
1           12     19
● (.venv) PS C:\PythonML\practica_red_neuronal_vscode>
```